

Research Report

Advanced Threat Monitoring for RobotLab

Fontys University of Applied Sciences

Course:	AV-CSP-AV-CSP1
Authors:	Team 5
Date:	September 16, 2026
Version:	1.0

Version History

Version	Date	Description
1.0	Week 16	Final version, end of Sprint 4

Contents

Version History	i
Summary	iv
1 Introduction	1
1.1 Origin, Background and Goal	1
1.2 Reading Guide	1
2 Project Description	2
2.1 Problem Description	2
2.2 Project Goal	2
2.3 Outcome Definition	3
2.4 Main Research Question	4
3 Research Approach	4
3.1 The DOT Framework	4
3.2 Way of Working	5
4 Preliminary Research	5
4.1 Literature and Internet Research	5
4.1.1 Threat Monitoring Concepts: XDR, SIEM and SOAR	5
4.1.2 Network Detection: NIDS, NSM and Deep Packet Inspection	6
4.1.3 Host and Runtime Detection: eBPF, HIDS and Enforcement	7
4.1.4 SDN and Cloud-Native Network Control	8
4.1.5 Threat Intelligence Sharing	9
4.1.6 Tool Comparison and Stack Selection	11
4.1.7 Standards, Regulations and Compliance	12
4.2 Requirements Analysis: Current and Desired Situation	16
5 Results	18
5.1 Analysis	18

5.2	Design: Reference Architecture	18
5.3	Realisation	20
5.4	Testing and Validation	22
6	Discussion and Evaluation	23
7	Conclusions	25
7.1	Conclusions per Research Question	25
7.2	Next Steps and Future Work	27
	List of Sources	29
A	Project Plan / Research Plan	32
B	Individual Tool Research Reports	32
C	Comparative Analyses per Tool Category	32
D	Comparative Analysis vs. Security Onion + MISP	33
E	Attack Simulation Run Book and Detection-Coverage Table	33
F	Pre-Production Gap Register	34
G	Network and Architecture Diagrams	35

Summary

RobotLab runs an automated, cloud based research environment on AWS. At the start of this project its threat detection relied on AWS GuardDuty, which only looks at high level logs. It does no deep packet inspection, it has no view inside the Kubernetes containers at runtime, and it gives little control over the security data, which also becomes expensive as the environment grows. The goal of this project was to research, design and build a more advanced, self hosted monitoring system that closes these gaps and gives RobotLab deeper visibility and more control.

The main research question was: How can an advanced hybrid monitoring system, integrating HIDS and NIDS, and leveraging SDN for dynamic control, be designed and implemented to protect RobotLab's AWS and on premise infrastructure from advanced cyberattacks?

To answer this question we followed the DOT framework, so every question was studied from more than one side: literature research, analysis of the real RobotLab environment, lab testing, and validation through stakeholder feedback and attack simulation. We worked in an agile way with sprints, starting with research and design and moving towards implementation and testing, which let us adjust the solution when new findings or problems came up.

The result is a reference architecture and a working implementation on AWS. The environment is split into two separate networks, one for the monitoring backend and one for the production Kubernetes cluster. On every node of the cluster four sensors run together: Suricata and Zeek for the network layer, and Falco and Tetragon for the runtime layer. All of their data is collected and correlated centrally in Wazuh, threat intelligence from MISP feeds the network sensors, and Tetragon can block a set of attacker tools directly in the kernel. We validated the system with a purple team exercise, where real attacks were run against the live environment and mapped to the MITRE ATT&CK framework. The system processed hundreds of thousands of alerts and detected or blocked most of the attacks, while the test also honestly showed where the blind spots are.

Our conclusion is that an advanced, self hosted hybrid monitoring system is feasible for RobotLab and clearly improves on the original GuardDuty only situation, for a predictable monthly cost. The dynamic network control that the question describes as SDN is provided by the eBPF based plugin Cilium instead of a classic SDN controller. Not everything was fully reached: the automatic response is only partly working, the cloud account logs were not yet being ingested during the test, and the on premise part was out of scope. Our main advice is to finish the response automation, repair the cloud monitoring, close the encrypted traffic blind spot, and make the central SIEM highly available. With these steps the system can grow from a strong detection platform into a complete detection and response solution.

1 Introduction

1.1 Origin, Background and Goal

This report describes the results of the project Advanced Threat Monitoring for RobotLab, carried out by Team 5 of Fontys University of Applied Sciences. RobotLab operates an automated, cloud-based research environment hosted on Amazon Web Services (AWS), in which interconnected systems, services, and processes handle potentially sensitive research data.

At the start of the project, RobotLab's threat detection relied primarily on AWS GuardDuty. While GuardDuty provides a baseline level of visibility, it operates on high-level log sources and managed findings only. It performs no deep packet inspection of network traffic, offers no visibility into runtime behaviour inside Kubernetes workloads, and provides limited customization of detection logic. As the environment grows, the cost of managed cloud security services also increases, while control over security data remains limited.

This project is a Blue Team focused research project: its purpose was to investigate how threat monitoring in this environment can be substantially improved through detection, monitoring, correlation, and response. The goal was to research suitable monitoring methods, tools, and techniques, to design a reference architecture for advanced threat monitoring in RobotLab, and to implement the most suitable solution or selected components of it. The research phase guided the implementation, so that technical decisions could be justified by evidence rather than preference.

1.2 Reading Guide

The remainder of this report is structured as follows. Chapter 2 describes the project itself: the problem RobotLab faces, the project goal, the intended outcomes, and the main research question with its five sub-questions. Chapter 3 summarizes the research approach, explaining how the research and development activities were set up following the DOT Framework and an Agile, sprint-based way of working.

Chapter 4 presents the preliminary research. It contains the results of the literature and internet research on threat monitoring concepts, candidate tools, and applicable standards and regulations, and concludes with the requirements analysis comparing the current and the desired situation. Chapter 5 then presents the results of the research and development activities in four parts: the analysis of RobotLab's infrastructure, the design of the reference architecture, the realised implementation, and the testing and validation of the implemented solution.

Chapter 6 discusses and evaluates these results, reflecting on their meaning and value for RobotLab as well as their limitations. Chapter 7 closes the main body of the report

by answering the research questions, critically evaluating what has been achieved, and recommending next steps for future research and development.

The report ends with the list of sources, cited in IEEE style, and the appendices, which contain the project plan, the individual tool research reports, the comparative analyses, and the validation evidence referenced throughout the text.

2 Project Description

2.1 Problem Description

RobotLab’s security monitoring depended on AWS GuardDuty as the primary detection mechanism. GuardDuty analyses AWS-native telemetry such as CloudTrail events [40], VPC Flow Logs, and DNS query logs, and produces managed findings [38]. Three structural limitations follow from this:

1. **No deep packet inspection.** GuardDuty observes flow metadata, not packet contents. Attacks that hide inside permitted network traffic, such as command-and-control channels, data exfiltration over DNS, and exploitation attempts inside HTTP, can remain invisible.
2. **No runtime visibility.** RobotLab runs containerized workloads on Kubernetes (Amazon EKS) [39]. GuardDuty provides no insight into what happens inside those workloads at the system-call level: process execution, privilege escalation, container escape attempts, and file access go unobserved.
3. **Limited control and rising cost.** Detection logic in a managed service cannot be freely customized, security data remains inside the provider’s service, and costs scale with the size of the environment.

For an environment in which attackers may target both infrastructure and sensitive research data, high-level detection alone means malicious activity can be missed. RobotLab therefore needed research into how it could move towards a more comprehensive, self-hosted monitoring architecture that provides deeper visibility, better detection, and more control over its security data.

2.2 Project Goal

The goal of the project was to research how an advanced threat monitoring environment can be built for RobotLab and to design a suitable approach for it. The project investigated which monitoring methods, tools, and techniques provide stronger detection capabilities and better visibility than the current situation, worked towards a reference

architecture for advanced threat monitoring, and implemented the most suitable approach on RobotLab's AWS infrastructure.

2.3 Outcome Definition

Ideal outcome. A fully operational, self-hosted XDR (Extended Detection and Response) platform covering all layers of the environment: network detection, host and runtime detection, cloud-account telemetry, central correlation, threat-intelligence integration, and automated response.

Defined products.

1. A comparison of relevant monitoring tools, methods, and techniques;
2. A reference architecture for advanced threat monitoring in RobotLab;
3. An implementation of the most suitable solution, or of selected components of that solution;
4. Documentation of the research process, technical choices, and obtained results;
5. Final conclusions and recommendations for future improvements.

MVP. The minimum viable product was defined as the reference architecture plus a working implementation of the core detection pipeline: network and host sensors on the production nodes, delivering telemetry to a central correlation point, validated with at least one controlled attack scenario.

What was delivered. All five products were delivered, and the detection pipeline exceeds the MVP. All four sensor types (Suricata, Zeek, Falco, and Tetragon) run on every node of the production cluster as DaemonSets. All telemetry is centrally correlated in Wazuh. Threat intelligence from MISP was integrated with the network sensors and demonstrated during the project: indicators of compromise are converted into Suricata rules and Zeek intelligence entries on an hourly cycle. In addition, in-kernel enforcement actively blocks a defined set of attacker tools on all nodes. For the response layer, a SOAR platform (Splunk SOAR) was deployed and connected to Wazuh, and a set of AWS Lambda response functions, such as isolating a compromised instance or stripping attacker permissions, was built and tested. The full automated chain, in which an alert triggers a SOAR playbook that runs the response on its own, was not yet activated within the project period. This remaining step, together with the operational state of the MISP integration at the end of the project, is discussed in Chapters 6 and 7.

2.4 Main Research Question

How can an advanced hybrid monitoring system, integrating HIDS and NIDS, and leveraging SDN for dynamic control, be designed and implemented to protect RobotLab's AWS and on-premise infrastructure from advanced cyberattacks?

Sub-questions.

1. How can the monitoring system combine kernel-level visibility through eBPF, runtime monitoring, and deep packet inspection to achieve effective threat detection across the infrastructure?
2. How can the monitoring system automatically respond to detected threats in order to contain attacks and minimize damage?
3. How should the defense layers be structured and distributed throughout the system to ensure both efficiency and security?
4. How can the attack surface be examined and how can the solution be validated?
5. How can data from HIDS and NIDS be correlated to accurately detect cyberattacks within the infrastructure?

The results in Chapter 5 and the conclusions in Chapter 7 are structured along these five sub-questions.

3 Research Approach

3.1 The DOT Framework

For this project we used the DOT (Development Oriented Triangulation) Framework, which is the standard research methodology for ICT projects at Fontys [1]. The main idea behind the framework is triangulation: every research question should be answered with more than one method, coming from different research strategies, so the findings can be checked against each other and the conclusions become stronger.

We used all five DOT strategies during the project:

1. **Library:** studying literature, technical documentation, and existing products. This gave the theoretical background on threat monitoring and the comparison of candidate tools. The results are in Chapter 4 and in the individual tool reports in Appendix B.

2. **Field:** analysing the current situation at RobotLab: the existing detection with GuardDuty, the AWS infrastructure, and the data flows. The results are in Sections 4.2 and 5.1.
3. **Lab:** testing, first of each monitoring component separately and later of the whole integrated system end-to-end. The results are in Section 5.4.
4. **Showroom:** presenting our work for feedback: peer review inside the team, expert review with the course instructors and RobotLab stakeholders, and demos during the sprints.
5. **Workshop:** hands-on exploration: building prototypes, researching design patterns, and doing a gap analysis between the current and the desired situation.

The complete mapping of strategies and methods per sub-question can be found in the Research Plan (Appendix A).

3.2 Way of Working

The project followed an Agile, sprint-based methodology. The early sprints concentrated on research: literature review, exploration of advanced threat monitoring concepts, and analysis of the RobotLab environment and its monitoring requirements. The middle sprints focused on the comparison of tools and techniques and on the design of the reference architecture. The later sprints shifted to implementation, testing, validation, and evaluation. At the end of each sprint the results were reviewed to determine what had been achieved and what the priorities for the next sprint were.

In practice these phases did not run strictly one after the other. Research and implementation overlapped: findings from early prototypes fed back into design decisions, and several architectural choices were revised when implementation revealed problems that the literature had not predicted. This was expected, and it is the reason an Agile approach was chosen. The most significant of these revisions are reported as results in Chapter 5 and reflected upon in Chapter 6.

4 Preliminary Research

4.1 Literature and Internet Research

4.1.1 Threat Monitoring Concepts: XDR, SIEM and SOAR

Before choosing tools, we first researched the main concepts behind modern threat monitoring, because the terms SIEM, XDR and SOAR are often mixed up while they each solve a different part of the problem.

A SIEM (Security Information and Event Management) is the classic central system. It collects logs from many different sources, normalises them, and correlates them against detection rules to raise alerts. Its strength is that everything ends up in one place where it can be searched and analysed. Its weakness is that a SIEM is log centric: it depends on what is sent to it, and it usually only detects and alerts, it does not act.

XDR (Extended Detection and Response) is a newer idea that grew out of the limits of EDR (Endpoint Detection and Response), which only looks at endpoints [12]. XDR extends detection across multiple layers, network, host, container and cloud, and correlates this telemetry across the layers so that related events become a single incident instead of many separate alerts. The goal of XDR is detection quality: fewer but more meaningful alerts, often mapped to a framework like MITRE ATT&CK [11].

SOAR (Security Orchestration, Automation and Response) solves the other half of the problem: what to do once an alert is raised [13]. A SOAR platform takes alerts and runs playbooks, which are automated workflows that can act across many different tools through their APIs. Where XDR improves the detection, SOAR improves the response, by reducing the manual work and shortening the time between an alert and an action.

From the literature it became clear that these are not competing choices but complementary layers. The mature pattern is to combine them: a SIEM/XDR core for high quality detection and correlation, and a SOAR layer for automated response orchestration. This is exactly the direction we took in the architecture. Wazuh fills the SIEM and XDR role as the central correlation point for all sensor data, and Splunk SOAR [16] is intended to fill the response orchestration role on top of it. The concrete implementation of this pipeline, including the Wazuh to SOAR integration and how far the automated response was actually realised within the project, is described in Section 5.3.

4.1.2 Network Detection: NIDS, NSM and Deep Packet Inspection

The first gap we identified in Section 2.1 was the missing deep packet inspection. Because of that, we researched how detection on the network layer actually works and which approaches exist. From the literature, two categories came forward: network intrusion detection systems (NIDS) and network security monitoring (NSM).

A NIDS inspects network traffic in real time and raises an alert when the traffic matches a known attack pattern. The most common approach is signature based detection. The system looks inside the packets (deep packet inspection) and compares the content against large rule sets that describe known malicious traffic, for example exploit attempts, malware command and control connections, or port scanning. The main tool we researched in this category is Suricata [18]. It checks traffic against thousands of community maintained rules, such as the Emerging Threats Open rule set, and writes structured alerts that a SIEM can read. The strength of this approach is that it is fast and precise for known threats. The weakness is that it cannot detect anything that has no signature yet.

NSM works the other way around. Instead of only alerting on known bad traffic, it records everything that happens on the network and turns it into structured logs. The tool we researched here is Zeek [19]. Zeek passively analyses the traffic and produces logs per protocol: every connection (conn.log), every DNS query (dns.log), every HTTP request (http.log), TLS certificates, transferred files, and strange behaviour it cannot classify. With these logs it becomes possible to detect behaviour that signatures cannot describe, like command and control beaconing, DNS tunneling [20], or lateral movement between systems. Zeek also has an intelligence framework that compares all observed traffic against a list of known bad indicators. This is the hook we later used for the MISP integration (Section 4.1.5).

During the research it became clear that these two approaches are not competitors but a pair. Suricata answers the question "is this traffic known to be bad?", and Zeek answers the question "what exactly happened on the network?". A Suricata alert is the starting point of an investigation, and the Zeek logs are the evidence to investigate it. Existing products like Security Onion [36] combine exactly these two tools, which confirmed for us that this is a proven pattern.

There are two limitations of network detection in a cloud environment that we have to mention. First, classic NIDS and NSM setups assume there is one central point in the network where all traffic can be captured. In a cloud and Kubernetes environment this point does not exist, so the sensors need to be spread over the infrastructure. This constraint had a big influence on our architecture in Section 5.2. Second, more and more traffic is encrypted, which limits what deep packet inspection can see. Techniques based on metadata, such as connection patterns, DNS queries and TLS certificate properties, keep working on encrypted traffic [21]. This made the combination with Zeek even more valuable.

The detailed research on Zeek, including the integration with Suricata and Wazuh, can be found in Appendix B. The comparison with alternative tools (Snort 3, Arkime, ntopng) is in Appendix C.

4.1.3 Host and Runtime Detection: eBPF, HIDS and Enforcement

The network detection stack is used for monitoring traffic between machines, but this leaves a blind spot once an attacker runs binaries within the host. In order to mitigate this situation, we need visibility on the host level.

eBPF (extended Berkeley Packet Filter) is currently the industry standard for runtime security tools in cloud and container environments because it provides a small set of verified programs to be attached to the Linux Kernel without installing any additional kernel modules [22]. These programs can monitor every system call with very low latency, making it ideal for our use case.

For this project we researched two eBPF-based tools for host and runtime detection,

Falco and Tetragon. Falco is a cloud-native runtime security tool that detects suspicious behavior in containers, Kubernetes, and Linux hosts by monitoring system calls and applying real-time security rules [26]. Even though Falco is flexible and easy to extend with custom rules it does not enforce or block any execution on the system, making it useful only for sending alerts.

In order to enforce rules upon the host and block any tentative of executing malicious binaries or scripts, we use Tetragon [27]. Tetragon can kill processes before they have time to execute, this removes TOCTOU (time-of-check-to-time-of-use) that every user-space tool has.

Kernel sensors produce only raw events. To turn events from many hosts into useful alerts, we use HIDS (host intrusion detection system). For this project Wazuh agent is used as log collector, file integrity monitoring, and configuration assesment [29]. The agent sends logs produced by other sensors to Wazuh manager. The manager matches the logs using detection rules, and the indexer (OpenSearch) stores the alerts. Wazuh is used this way as SIEM for the arhitecture, a place where we can follow every log that the sensors generate.

4.1.4 SDN and Cloud-Native Network Control

Software-Defined Networking (SDN) separates the control plane of the network from its data plane [32]. An central controller is used for complete view of the network and programs forwarding rules of the switches. Policies can be defined centrally and changed dynamically, which can be used for security because the network can react to threats.

In this project SDN cannot be used because our environment is in public cloud. The cloud provider's network is not accessible, AWS's network by itself is software defined and the customer gets only VPC abstraction with their controls such as: security groups, network ACLs, route tables. A central SDN controller will be a single point of failure [33], which is not a good fit for a Kubernetes environment where IP's and workloads changes constantly.

Instead of classic SDN, this project uses Cilium [23], more compatible with cloud environments. Cilium is based on eBPF and runs its data plane directly in the kernel of each node. Network rules are written as CiliumNetworkPolicy objects and are based on labels instead of IP addresses [25], which helps when pods are shord-lived and IPs are changing. Each node enforces these rules locally so there is no central controller and no single point of failure.

Cilium brings Hubble [24], tool that give network visibility into the traffic between workloads, and WireGuard [28] encrypts traffic between nodes. In this way the network is still programmable, policies can be defined centrally, and the network can still react to threats.

4.1.5 Threat Intelligence Sharing

Effective threat monitoring does not operate in isolation: the value of a detection system depends in part on the quality and freshness of the threat data it acts on. Threat intelligence sharing addresses this by providing a structured mechanism for collecting, storing, and distributing Indicators of Compromise (IoCs) and broader adversarial context across organisations and communities.

The problem of unstructured sharing. Before dedicated platforms existed, threat data was exchanged informally through emails and PDF reports. This approach makes automated processing difficult: a NIDS cannot ingest a PDF, and a SIEM cannot correlate against an inbox. The solution is a platform that stores threat data in a structured, machine-readable format and exposes it through standardised APIs and export formats.

MISP — Malware Information Sharing Platform. MISP is the open-source threat intelligence platform selected for this project. It was originally developed by CIRCL (Computer Incident Response Center Luxembourg) and has been in production use since 2011 [34]. It is governed under an open-source, community-driven model with EU funding support, and its data format is formalised as an IETF Internet-Draft, ensuring interoperability beyond the platform itself.

The fundamental data unit in MISP is an *event*, representing a specific threat or incident report. Events contain *attributes* – individual IoCs such as malicious IP addresses, file hashes, and domain names – of which MISP supports over 170 types across 16 categories [35]. Related attributes are grouped into *objects* (over 300 templates available), and events are tagged with *taxonomies* (standardised classification systems including TLP, MITRE ATT&CK, and GDPR categories) and *galaxies* (curated knowledge bases of threat actors, malware families, and attack techniques).

Correlation and automation. MISP’s correlation engine automatically links newly ingested attributes against existing data using exact matching, fuzzy hashing (ssdeep), and CIDR block matching. This turns a repository of raw IoCs into a connected graph of related indicators. For consumption by downstream tools, MISP can export directly to Suricata and Snort rule format, Zeek/Bro intelligence format, STIX 1.x/2.x, YARA, Sigma, and several other formats. A REST API (documented in OpenAPI format) and the PyMISP Python library allow fully automated pull/push workflows, and scheduled feeds can be configured to synchronise on a defined interval.

Integration with the monitoring stack. In the reference architecture, MISP fills the threat intelligence layer: IoCs from MISP are converted into Suricata rules and Zeek intelligence entries on an hourly cycle and pushed to the network sensors. This closes the

loop between community threat intelligence and active network detection without manual intervention.

Sharing controls and privacy. Because sharing threat data involves sensitive organisational information, MISP provides layered access controls. Distribution can be set at the individual attribute level using Traffic Light Protocol (TLP) and Protection Against Publication (PAP) tags. Custom sharing groups allow organisations to define specific trusted audiences. Delegated sharing supports pseudo-anonymous publication, and sensitive notifications are encrypted with GnuPG or S/MIME. MISP also provides compliance guidance for GDPR, ISO/IEC 27010:2015, and the EU NIS Directive.

Community and ecosystem. MISP’s practical value is amplified by its community. Table 2 lists a selection of known public sharing communities; many more private communities exist. For this project the relevant feeds are the CIRCL OSINT feed and any community feeds relevant to cloud and container attack techniques.

Table 2: Selected public MISP sharing communities

Community	Sector	Notes
CIRCL MISPPRIV	Private sector	1,100+ organisations
CIRCL Financial	Finance	Banks and payment processors
NATO MISP	Defence	NATO member nations
FIRST MISP	Multi-sector	Global incident response community
MISP-LEA	Law enforcement	International LEA sharing
ICS-CSIRT.io	Industrial	ICS/OT threat data

Comparison with alternatives. MISP was selected over OpenCTI, TheHive, and commercial alternatives. OpenCTI [37] offers native STIX 2.1 support and a modern stack but has a smaller community and less mature sharing infrastructure. TheHive is primarily a case management and incident response platform with limited native sharing capability. Commercial platforms offer stronger analytics but introduce vendor lock-in and significant licensing cost. MISP’s combination of free licensing, best-in-class sharing and synchronisation, broad format support, and the largest open community makes it the most practical choice for the reference architecture.

Limitations. The main operational risks associated with MISP are setup and maintenance effort, performance degradation at scale without database tuning, and data quality that depends on community contributions. Within the project, the MISP integration was

demonstrated operationally – IoCs were converted to Suricata rules and Zeek intel entries on the defined schedule – but the synchronisation channel had a TLS validation gap at the end of the project period, which is documented as GAP-002 in the Pre-Production Gap Register (Appendix F) and discussed further in Chapter 6.

4.1.6 Tool Comparison and Stack Selection

After the literature research into the individual concepts, we compared concrete tools for every layer of the planned architecture. The full comparison, with all evaluated alternatives per category, is in Appendix C; this section summarizes how we selected and what came out.

Every tool had to meet the same selection criteria before it was considered:

1. Fully open-source with an OSI-approved license, so RobotLab has no licensing costs and keeps full control over the software;
2. Portable to bare metal, because a future on-premise environment should be able to reuse the same stack;
3. Kubernetes-native or Kubernetes-aware, since the production workloads run on EKS;
4. An active community and ongoing development, so the tool will still be maintained in a few years;
5. Good integration with the rest of the stack, because the value of the architecture comes from correlation, not from individual tools.

With these criteria we evaluated alternatives in each category. The most important choices are summarized in Table 3.

Table 3: Tool selection per architecture layer

Category	Chosen	Main alternatives considered
Network IDS (deep packet inspection)	Suricata	Snort 3
Network security monitoring	Zeek	Arkime, ntopng
SIEM / XDR core	Wazuh	Elastic Security, Security Onion, OS-SIM, TheHive + Cortex
Container runtime detection	Falco	Tracee, KubeArmor
eBPF enforcement	Tetragon	bpfftrace, Inspektor Gadget
CNI / network control	Cilium	Calico, Flannel, Weave
Threat intelligence platform	MISP	OpenCTI, Yeti, CRITs

A few of these choices deserve a short explanation. For the SIEM core we chose Wazuh over Elastic Security mainly because of cost and simplicity: Elastic is technically very capable, but its more advanced security features sit behind paid subscription tiers [15], while Wazuh offers agents, correlation rules, and dashboards completely free [30]. For network detection we did not have to choose between Suricata and Zeek, the literature research (Section 4.1.2) showed they complement each other, with Suricata matching known attack signatures and Zeek recording context around all traffic. The same applies to Falco and Tetragon: Falco is the de facto standard for runtime detection, while Tetragon can also enforce in the kernel, so we use both.

We also compared our composed stack as a whole against Security Onion, an existing integrated distribution that bundles Suricata, Zeek, and Wazuh into one product. Security Onion would have saved integration work, and its full packet capture is stronger for forensics. However, it is designed for traditional networks with a central traffic tap, not for a cloud-native Kubernetes environment: it has no runtime visibility inside containers, no admission or network policy enforcement, and no straightforward deployment path on EKS. In our own comparative analysis (Appendix D), we mapped both options against the MITRE ATT&CK techniques relevant for RobotLab, and the composed stack covered roughly twice as many techniques as Security Onion with MISP. This confirmed the decision to compose the stack ourselves, even though it costs more integration effort.

4.1.7 Standards, Regulations and Compliance

The following standards and frameworks were identified as applicable to the Advanced Threat Monitoring project. They cover the full technical scope of the project and are complemented by Dutch national standards and legislation applicable to a publicly funded

university of applied sciences.

NIST SP 800-137 – Information Security Continuous Monitoring. NIST SP 800-137 defines a framework for establishing an information security continuous monitoring (ISCM) programme [2]. It provides guidance on defining metrics, implementing monitoring solutions, analysing data, and responding to findings. The publication’s emphasis on automating data collection and establishing monitoring tiers (organisation, mission/business process, and information system levels) directly informed the structure of the reference architecture. It provides the baseline definition of what continuous monitoring means in a cloud and hybrid environment, and informs the metrics and coverage criteria used to validate monitoring sufficiency.

NIST SP 800-94 – Guide to Intrusion Detection and Prevention Systems. NIST SP 800-94 provides comprehensive guidance on the design, deployment, and management of Intrusion Detection and Prevention Systems (IDPS), covering network-based, host-based, and hybrid approaches as well as detection methodologies including signature-based, anomaly-based, and stateful protocol analysis [3]. As the main research question explicitly focuses on integrating HIDS and NIDS components, SP 800-94 serves as the primary theoretical and architectural reference for the HIDS/NIDS design decisions in Sub-questions 1 and 5, informs the selection and comparison of tools such as Suricata, Zeek, and Falco, and provides the detection methodology taxonomy used in the tool comparison matrix.

CIS Controls v8. The CIS Controls v8 defines a prioritised set of security actions designed to mitigate the most common attack vectors [7]. Control Group 8 (Audit Log Management), Control Group 13 (Network Monitoring and Defence), and Control Group 10 (Malware Defences) are of particular relevance. CG 8 informs the log pipeline design and retention policies; CG 13 provides implementation safeguards against which the NIDS component can be evaluated. CIS Controls v8 is used as a practical checklist during the validation phase to verify that the implementation covers expected monitoring baseline behaviours.

MITRE ATT&CK Framework. The MITRE ATT&CK framework is a globally recognised knowledge base of adversary tactics, techniques, and procedures (TTPs), structured across Enterprise, Cloud, and Containers matrices [11]. The validation strategy calls for mapping detection coverage against a threat framework; ATT&CK is the industry standard for this purpose. It is used in the Detection Coverage Analysis to document which techniques are detected, partially detected, or missed, informs the controlled attack simulation scenarios defined in Sub-question 4, and provides the attack surface enumeration basis for identifying threats most relevant to RobotLab.

ISO/IEC 27035 – Information Security Incident Management. ISO/IEC 27035 defines a structured process for planning and managing information security incidents, covering detection, reporting, assessment, response, and lessons learned [8]. Sub-question 2 specifically addresses how the monitoring system can automatically respond to detected threats in order to contain attacks and minimise damage. ISO/IEC 27035 provides the process model within which automated and manual response actions are structured, informs the automated response workflow design and containment steps triggered through the eBPF-based network control (Cilium) and the SOAR response layer, and defines the scope and expected outputs of the incident response component in the reference architecture.

NIST SP 800-190 – Application Container Security Guide. NIST SP 800-190 addresses security risks associated with container technologies, including image vulnerabilities, container runtime threats, host OS exposure, and Kubernetes security [4]. RobotLab’s infrastructure runs on Kubernetes, and one of the identified gaps in the current monitoring setup is the lack of visibility into Kubernetes runtime behaviour. SP 800-190 directly addresses this gap, is used to identify monitoring requirements for the container and Kubernetes layers (Sub-question 1), provides the container security risk model for the infrastructure analysis phase, and informs the eBPF-based runtime monitoring design in relation to container escape and lateral movement detection.

ISO/IEC 27001 Annex A – Monitoring-Related Controls. ISO/IEC 27001 Annex A contains specific technical controls directly relevant to monitoring: A.8.15 (Logging), A.8.16 (Monitoring Activities), and A.8.20 (Networks Security) [9]. These provide a widely accepted baseline for what a monitoring system must log, retain, and alert on. A.8.15 and A.8.16 inform the minimum logging and alerting requirements of the architecture; A.8.20 is applied during the infrastructure analysis to assess existing network security visibility gaps; and the controls are used in the final evaluation to frame project outcomes in terms of a recognisable compliance baseline.

IEC 62443 – Industrial Automation and Control Systems Security. IEC 62443 defines requirements and processes for IACS security across a layered model of zones and conduits [10]. The most relevant parts are IEC 62443-2-1 (security management requirements), IEC 62443-3-3 (system-level security requirements), and IEC 62443-4-2 (component-level security requirements). In the Netherlands this series is adopted as NEN-EN-IEC 62443. RobotLab’s orchestrator-to-instrument communication constitutes a control system topology within the scope of this standard. The zone-and-conduit model provides a structured basis for separating the IT monitoring plane from the OT control plane and for defining security requirements at each boundary. IEC 62443-3-3 system requirements (SR 6.1, SR 6.2) inform the monitoring and audit log requirements for the orchestrator and instrument communication channel.

NIST SP 800-82 – Guide to Operational Technology Security. NIST SP 800-82 provides guidance on applying security controls to OT environments, addressing the availability, safety, and timing constraints that differentiate OT security from IT security [5]. RobotLab’s hybrid IT/OT topology – including the lab facility, physical instruments operated via the orchestrator, and Grafana-based monitoring – introduces monitoring challenges not covered by IT-focused standards alone. SP 800-82 informs the monitoring strategy for the orchestrator-to-instrument plane, provides OT-specific incident response guidance to complement ISO/IEC 27035, defines the monitoring boundary between the IT and OT layers, and informs risk analysis for scenarios where a compromised orchestrator could have physical consequences.

Baseline Informatiebeveiliging Overheid 2 (BIO2). BIO2 (version 1.3, published in the *Staatscourant* on 5 March 2026) is the common information security baseline for Dutch government. It is structured according to NEN-EN-ISO/IEC 27001:2023 and NEN-EN-ISO/IEC 27002:2022 and has been strengthened to align with NIS2 requirements. Because BIO2 follows the structure of NEN-EN-ISO/IEC 27002:2022, it directly incorporates the monitoring-related controls 8.15 (Logging), 8.16 (Monitoring activities), and 8.20 (Networks security). BIO2 provides the Dutch national operationalisation of the logging and monitoring baseline already referenced through ISO/IEC 27001 Annex A, and its risk-based approach informs the selection of monitoring measures beyond baseline controls.

Cyberbeveiligingswet (Cbw) – Dutch Implementation of NIS2. The Cyberbeveiligingswet is the Dutch transposition of the NIS2 directive, adopted by the Tweede Kamer on 15 April 2026 and awaiting Senate approval, with entry into force expected around 1 July 2026. Notably, the Netherlands extended scope beyond the NIS2 minimum to explicitly include universities of applied sciences. The duty of care (*zorgplicht*), incident reporting duty (*meldplicht*), and registration duty (*registratieplicht*) create concrete legislative drivers for continuous monitoring and reliable incident detection. The incident reporting duty directly informs the detection-to-reporting timeline that the automated response workflow must support, and complements ISO/IEC 27035 with a national legal basis.

NCSC Guidance on OT/IACS Monitoring. The NCSC publishes practical national-context guidance for securing OT/IACS environments, recommending passive monitoring within OT networks – for example through port mirroring – so that monitoring does not disrupt operational continuity or physical safety. This guidance is the Dutch national-context counterpart to the OT-safe monitoring constraints derived from NIST SP 800-82 and IEC 62443, and reinforces the constraint that automated containment must not jeopardise physical lab operations or safety.

Standards mapping. Table 4 provides a concise overview of how each standard maps to the research sub-questions.

Table 4: Standards mapping to research sub-questions

Standard	Applicable Sub-questions
NIST SP 800-137	SQ1, SQ3, SQ5
NIST SP 800-94	SQ1, SQ3, SQ5
CIS Controls v8	SQ1, SQ3, SQ4
MITRE ATT&CK	SQ4
ISO/IEC 27035	SQ2, SQ4
NIST SP 800-190	SQ1, SQ3
ISO/IEC 27001 Annex A	SQ3, SQ5
IEC 62443 (2-1, 3-3, 4-2)	SQ1, SQ2, SQ3
NIST SP 800-82	SQ1, SQ2, SQ3, SQ4
BIO2	SQ3, SQ5
Cyberbeveiligingswet (Cbw / NIS2)	SQ2, SQ4
NCSC OT/IACS guidance	SQ1, SQ2, SQ3

4.2 Requirements Analysis: Current and Desired Situation

Besides the literature research we also had to look at RobotLab itself. The general problem from Section 2.1 was still quite abstract, so we needed to translate it into actual requirements that we could design and build against. This was the Field part of the research. In practice it meant studying the AWS environment and the monitoring that was already there, and writing down where the gap was between that and what RobotLab actually needed.

The situation when we started was as follows. Everything runs on AWS, the production applications run on a Kubernetes cluster (Amazon EKS). The only real security monitoring was AWS GuardDuty, which uses log sources like CloudTrail and VPC Flow Logs. GuardDuty is fine for what it does, it catches suspicious account activity and some network anomalies on a high level. The problems start when you want more than that. It does not look inside network packets, it has no idea what happens inside a running container, you can not really write your own detection rules for it, and the data it collects stays inside the AWS service so you can not take it somewhere else to analyse it. We already described this in Section 2.1. In short, the account layer was covered, the network and runtime layers were not.

What we wanted instead was an environment where those layers are also covered, without breaking what already worked. So detection on the network itself, both with signatures and with behaviour based monitoring. Visibility inside the containers, on the level of processes and system calls. Threat intelligence, so the sensors actually know which indicators to look for instead of only relying on static rules. And one central place where everything ends up, because logs spread over five different tools are useless for an investigation. We also wanted the system to be able to respond to an obvious threat on its own, not just raise an alert. An important condition from RobotLab's side was that all of this had to be self hosted, they wanted to own their security data, and the monthly cost had to stay predictable.

From this comparison, plus the standards we described in Section 4.1.7, we wrote down the requirements. The functional ones first, so what the system must be able to do:

1. Inspect network traffic for known attack patterns (deep packet inspection / NIDS)
2. Record network activity as structured logs, for investigation and behaviour based detection (NSM)
3. Monitor process and system call activity inside running containers (HIDS / eBPF)
4. Keep ingesting the cloud telemetry that was already there (CloudTrail, GuardDuty findings)
5. Have a threat intelligence platform that feeds indicators of compromise to the sensors
6. Bring all telemetry together in one central system for correlation, searching and dashboards
7. Take an automated response action when there is a clear threat

And the non functional ones, so the qualities the system should have:

1. Self hosted, RobotLab keeps full ownership of its security data
2. Open source tools, no licensing costs and no vendor lock in
3. Works in a distributed cloud environment, since there is no single point where all traffic passes (Section 4.1.2)
4. Portable to bare metal, because RobotLab plans to move on premise later
5. Reproducible, the whole environment must be rebuildable
6. Predictable monthly cost

These requirements are what the rest of the report hangs on. Section 5.2 describes the architecture we designed to meet them, and in Section 5.4 we test how well the built system actually does.

5 Results

5.1 Analysis

We started by analysing the AWS account of RobotLab. We needed this to know what we had to monitor. We looked at the VPCs, the workloads, the traffic going in and out, and the parts of the environment that an attacker can reach. We used the results for the design of the infrastructure and for the attack scenarios in Section 5.4.

During the analysis we decided to keep the monitoring separate from production. If the monitoring tools run in production and an attacker compromises production, the attacker can also reach the monitoring tools and remove the logs. The monitoring stack also contains credentials and information about the whole environment, so it has to be protected itself too. So we made two planes.

The control plane is the bc-ctrl VPC (10.0.0.0/16). The security workloads run here on EC2 instances: the central log collection, correlation and the threat intelligence platform. The production plane is the bc-prd VPC (10.30.0.0/16). The EKS cluster with the production applications runs here. The sensors run in the production plane, because the traffic and system calls are there. The data they collect is sent to the control plane. Section 5.2 describes which components run where.

We also made an inventory of the attack surface of the production plane. This is a list of the points where an attacker can get in or move around: the public endpoints of the applications, the Kubernetes API, the worker nodes, the containers and the AWS account itself (IAM credentials). In Section 5.4 we use this list to choose the attack simulations, because the system should be able to detect every point on the list.

5.2 Design: Reference Architecture

The reference architecture is the main design product of the project. It shows how the tools we selected work together to meet the requirements from Section 4.2. The full network diagram is delivered as a separate file (see Appendix G). In this section we go through the design layer by layer and explain the reasoning behind the most important choices.

The first decision we made was to split the environment into two separate networks (VPCs) that are connected by a private peering link. One network is the control plane, and it holds the monitoring backend: the central SIEM, the threat intelligence platform, and the build server. The other network is the production plane, and it holds the Kubernetes cluster with the workloads that we actually want to monitor. We did this mainly for security reasons. The systems that store and analyse all the security data are kept apart from the systems that run the workloads and that are exposed to attacks. So if something in the production plane gets compromised, the monitoring backend is not sitting in the

same network. This is also how we answer sub question 3 about how the defense layers should be structured. The two networks can still reach each other over the private peering link, and a private DNS zone lets the sensors find the SIEM by name instead of by a fixed address.

Inside the production cluster, every node runs a set of sensors as a DaemonSet, which simply means that one copy of each sensor runs on every node. This is our answer to the problem from Section 4.1.2, that in a cloud environment there is no single point where all the traffic can be captured. Instead of one central tap, we spread the sensors over the whole cluster. There are four sensor types, and each one covers a different layer:

1. **Suricata** does the deep packet inspection. It checks the traffic against signatures and raises an alert when it sees a known attack pattern.
2. **Zeek** does the network security monitoring. It records the traffic as structured logs, so we keep the context and can also detect behaviour that has no signature.
3. **Falco** watches what happens inside the containers on the system call level, and raises alerts on suspicious runtime behaviour.
4. **Tetragon** also works on the kernel level with eBPF, but it can do more than report. It can also enforce, which means it can actively stop a process instead of only alerting on it.

Together these four cover the network layer and the runtime layer that were blind in the old situation, and that is our answer to sub question 1.

For the network itself, the cluster uses Cilium as its network plugin. Cilium is built on eBPF and it lets us write network policies that say exactly which workload is allowed to talk to which other workload. This gives us the dynamic and programmable control over the network that the main research question describes as “SDN”, but in a way that actually fits a cloud native environment. The reason why we used eBPF instead of a classic SDN controller is explained in Section 4.1.4. On top of that, Cilium encrypts the traffic between the nodes with WireGuard, so the telemetry that travels between the systems is protected.

All the telemetry from the sensors is sent to one central system, Wazuh, which runs in the control plane. On every node a Wazuh agent collects the logs that the sensors write, and forwards them to the Wazuh manager. The manager decodes the events, applies the detection rules, and stores everything in its indexer, where it can be searched and shown on a dashboard. This is the one central place where the data from the host based and the network based sensors comes together, and that is our answer to sub question 5 about correlating HIDS and NIDS data.

The threat intelligence comes from MISP, which also runs in the control plane. In the design, the network sensors pull the current indicators of compromise from MISP on a

regular schedule and turn them into Suricata rules and Zeek intelligence entries. This way the threat intelligence from the community automatically becomes local detection, and nobody has to update the rules by hand.

To keep the cloud visibility that the old situation already had, the design also feeds the AWS account logs (CloudTrail and the GuardDuty findings) into Wazuh. So the central SIEM sees the deep network and runtime events from the cluster together with the high level account events from AWS, all in one place.

For the response part, the design works on two levels. On the runtime layer, Tetragon enforces directly: with a kernel-level policy, a defined set of attacker tools is blocked the moment they try to run, which gives an automatic response on the node itself. On top of that, the design adds an orchestration layer. A SOAR platform (Splunk SOAR) receives the high-severity alerts from Wazuh and runs playbooks that can invoke AWS Lambda response functions to act on the cloud side, for example isolating a compromised instance or stripping attacker permissions. In the design, the runtime enforcement and these response functions are the two halves of the automatic response. The runtime enforcement and the Lambda response functions were built and tested; activating the full automated chain, in which an alert triggers a SOAR playbook that runs the response on its own, is the remaining step. We come back to this in Chapters 6 and 7.

5.3 Realisation

After the design was finished, we built the system in practice. This section explains what we actually built and how, following the same layers as the design in Section 5.2.

We built the whole environment as Infrastructure as Code with Terraform. Both networks (VPCs), the Kubernetes cluster and all the supporting AWS resources are written in code instead of being set up by hand. The main reason for this is that the environment becomes reproducible, which was one of our non functional requirements: we can take everything down and build it back up again in a reliable way. The Kubernetes cluster runs on Amazon EKS with version 1.35, and the security stack on top of it is installed with Helm, the package manager for Kubernetes. Through Helm we deployed Cilium 1.19.3 as the network plugin, Falco 8.0.2, Tetragon 1.6.1 and the External Secrets operator. The four sensors (Suricata, Zeek, Falco and Tetragon) are deployed as DaemonSets, which means Kubernetes runs one copy of each sensor on every node.

On the control plane we set up the monitoring backend on normal virtual machines (EC2). The Wazuh manager, indexer and dashboard run together on one instance, MISP runs on another, and there is a build server for the deployment pipeline. One thing worth mentioning is that we first planned to run this backend inside Kubernetes too, but during the implementation we moved it to EC2, because running the SIEM on the same cluster that it is supposed to watch turned out to be fragile. We look back on this design change in Chapter 6.

The telemetry flows as follows. Each sensor writes its events to a log file on the node (a `hostPath` log). On every node the Wazuh agent, also deployed as a `DaemonSet`, reads those log files and forwards the events over the peering link to the Wazuh manager. The manager decodes the events, applies the detection rules and stores them in the indexer, from where they can be searched and shown on the dashboard.

To actually detect things, we wrote our own detection content. On the Wazuh side we added custom rule sets for the different sources, so the events from Suricata, Falco, Tetragon, Zeek and the Kubernetes audit log are turned into useful alerts. We also wrote detection rules for the AWS account logs (`CloudTrail`), but as we explain in Sections 5.4 and 6, the ingestion of those logs was not actually working at the end of the project, so those particular rules did not yet receive data. For the enforcement we wrote Tetragon `TracingPolicies`. The most important one is `sigkill-malicious-tools`, which kills a defined set of attacker tools (like `nmap` and `nc`) the moment they are started, and there is a second policy, `privileges-raise`, that watches for privilege escalation. For the threat intelligence we built small sidecar containers that run next to Suricata and Zeek and pull the indicators from MISP on a schedule, and then convert them into Suricata rules and Zeek intelligence entries.

For the secrets, like API keys and passwords, we did not put them in the code. We used AWS Secrets Manager together with the External Secrets operator, which uses an AWS role (IRSA) to pull the secrets into the cluster only when they are needed. On top of that, every namespace in the cluster has its own Cilium network policy, so the workloads can only talk to the things they are allowed to talk to. This follows the zero trust idea from the design in Section 5.2.

Finally, we automated the whole deployment in a CI/CD pipeline built with GitHub Actions. The pipeline can build the complete platform from an empty AWS account up to a running system, in the right order and without manual steps. It applies the resources in stages, checks for drift in the state, and can heal itself when a part is not ready yet. This was not easy, because many of the parts depend on each other and have to start in a specific sequence. Getting this cold start to work in a reliable way was one of the bigger pieces of engineering in the project, and the problems we ran into are discussed in Chapter 6.

On top of the detection pipeline we built the response layer. We deployed Splunk SOAR on the control plane and connected it to Wazuh, so that high-severity alerts are forwarded to the SOAR over its REST API. On the AWS side we wrote a set of Lambda response functions that carry out the actual defensive actions, such as isolating a compromised instance from the network, removing a node from the cluster, and stripping attacker IAM permissions. We tested these response functions by invoking them against test targets and confirmed that they perform the action as intended. The last link, a SOAR playbook that takes an incoming alert and triggers the matching Lambda on its own, is designed but was not yet activated within the project period, which we discuss in Chapters 6 and

7.

5.4 Testing and Validation

To find out if the system really works, we tested it the way we planned in the Research Plan. We did this as a purple team exercise. One of us played the attacker and another one watched the dashboards and wrote down for every attack whether it was detected, blocked, or missed. We mapped all the attacks to the MITRE ATT&CK framework, so we could measure the coverage in a structured way.

The first thing we saw is that the whole pipeline really works from start to end. During the test the system handled more than 215,000 alerts in 24 hours, all mapped to MITRE ATT&CK, and all four sensor agents were reporting into Wazuh. The runtime sensor (Tetragon) alone produced more than 45,000 events. So the main goal, getting the data from the sensors on the nodes all the way to the central dashboard, was reached.

Table 5 shows the most important attacks we ran and the result for each one.

Table 5: Executed attack scenarios and detection results

Attack (technique)	MITRE	Sensor	Result
Run nmap or nc inside a container	T1059	Tetragon	Blocked
Spawn a shell inside a container	T1059.004	Tetragon / Falco	Detected
Read service account token and exfiltrate it	T1552.001	Tetragon	Detected
Privilege escalation (setuid)	T1548	Tetragon	Detected
Container escape attempt	T1611	runc (patched)	Blocked
Pod tries to steal cloud credentials	T1552.005	Cilium	Blocked
Persistence with a Kubernetes CronJob	T1053.007	Wazuh k8s-audit	Detected
Port scan from a pod	T1046	Suricata	Detected
HTTP login brute force	T1110	Suricata	Detected
Denial of service floods (SYN/UDP/ICMP)	T1498	Suricata	Detected
Command and control on an odd port	T1071	Cilium	Blocked and alarmed
DNS tunneling out of a pod	T1048.003	Zeek / CoreDNS	Detected
Lateral movement between app services	T1210	Hubble	Visible, not blocked
Encrypted command and control over 443	T1071.001	Suricata / Zeek	Missed
Cloud control plane events (CloudTrail)	T1562	Wazuh aws-s3	Missed

The results show a clear pattern. The runtime layer (Tetragon and Falco) caught the activity inside the containers very well, and the enforcement policy kills the attacker tools the moment they start. The network detection (Suricata) caught the loud attacks that have a clear traffic pattern, like scanning, brute force, and the denial of service floods.

The escape and cloud protections also held, because the container could not break out and a pod could not steal the cloud credentials.

There were two blind spots that stayed open at the end. The first one is encrypted command and control over normal web traffic on port 443. Because the traffic is encrypted before the network sensors can read it, this kind of hidden channel is missed. The second one is the cloud control plane layer. The AWS account logs are supposed to be read into Wazuh, but during the test we found out that this part was not actually running, so attacks against the cloud account did not raise an alert. We come back to this in Chapter 6.

We also looked at how much noise the system makes, because a monitoring system that floods the analyst with useless alerts is not usable in practice. Most of the alerts were relevant, but we found one rule (the privilege escalation rule on `setuid`) that fired way too often and buried the more important alerts. This is a real false positive problem and it is on our list of things to improve.

The last step was to compare the result with the problem from Section 2.1. The deep packet inspection and the runtime visibility that GuardDuty did not have are now there, and we proved that they fire on real attacks. The data is also centralised and self hosted. But the test also showed honestly that not everything is covered. The encrypted network traffic and the cloud audit trail are still blind spots, and the automated response only works on the runtime layer. This comparison is the base for the conclusions in Chapter 7.

6 Discussion and Evaluation

In this chapter we look at what our results actually mean for RobotLab, what value they bring, and where the limitations are. We try to be critical here, because an honest view on the weak points is just as important as the things that worked.

Value of the result. The most important value is the extra detection depth compared to the old situation. In Section 2.1 the main problem was that GuardDuty only looked at high level logs, with no deep packet inspection and no view inside the containers. Our validation in Section 5.4 showed that both of these gaps are now really covered. The network sensors catch scanning, brute force, and denial of service attacks, and the runtime sensors catch and even block what happens inside the containers. On top of that, all of this data comes together in one central place, where events from different layers can be seen next to each other. So the system does what it was supposed to do: it gives deeper and broader visibility than the starting point.

A second value is the cost. The whole platform is self hosted and built from open source tools, so there are no licensing fees per agent or per gigabyte. The running cost is mainly the cost of the cloud infrastructure, which is predictable and stays around a few hundred

dollars per month. Commercial XDR and SIEM products usually charge based on the amount of data that is ingested, which can grow into very high yearly costs as the environment grows. For an environment like RobotLab, that keeps growing, a predictable self hosted cost is an advantage.

A third value is portability. Because the stack is open source and not tied to one cloud provider, it can in principle be moved to a bare metal or on premise environment later. This fits the future plans of RobotLab, even though that part was out of scope for this project.

Limitations. The first limitation is that the central SIEM runs on a single machine. This makes it a single point of failure: if that one machine goes down, the whole detection stops. This is not only a theoretical risk. During the test the sensor agents lost their connection to the SIEM and the dashboard went blind for a while, until we fixed it. A production setup would need a high availability design instead of one box.

The second limitation is in the cloud layer. In the design we planned to read the AWS account logs (CloudTrail and GuardDuty) into Wazuh, so the cloud visibility would not be lost. During the test we found out that this part was not actually working: the events were not being ingested, so attacks against the cloud account did not raise any alert. This means that, at the end of the project, the cloud control plane is the weakest monitored layer, even though the detection rules for it are already written.

The third limitation is the transport security between the components. The channel between the agents and the SIEM is not encrypted by itself, and the threat intelligence sync ignores the certificate check. The traffic between the nodes is protected by WireGuard, so it is not fully open, but these points are still real weaknesses and they are listed as known gaps that should be solved before production.

The fourth limitation is the automated response. This was sub question 2, and it is only partly answered. The runtime enforcement works well, because Tetragon kills a defined set of attacker tools in the kernel the moment they start. The orchestration layer is also in place: the SOAR platform is deployed, it receives the high-severity alerts from Wazuh, and the AWS Lambda response functions that it would call, for example to isolate a compromised instance or strip attacker permissions, were built and tested on their own. What is missing is the last step, activating the SOAR playbooks so that an alert triggers the response automatically. During the project this automated chain was not switched on, so in practice the cross-system response still had to be started by hand. The system is therefore strong in detection and already has automatic enforcement on the runtime layer, but the fully automatic orchestrated response across the whole system is the part that stayed unfinished.

There are also a few more specific limitations. The network sensors are blind to encrypted content, so command and control hidden inside normal web traffic on port 443 is not

detected. The runtime enforcement is based on the name of the program, which means an attacker could rename or copy a tool to get around it. And finally, the validation was done in one lab environment and over a short period, so there is no long term measurement of the false positive rate yet, and the on premise part of the main research question could not be tested because it was out of scope.

The process. Looking back at how we worked, the DOT framework and the agile way of working were useful. The combination of library research, lab testing, and the purple team validation gave us results from more than one angle, which is exactly the triangulation idea. The validation in particular was very valuable: it did not only confirm what worked, it also found real problems that we could then fix, which made the system better. The agile approach also helped us deal with changes during the project, like moving the SIEM from Kubernetes to virtual machines when the first design turned out to be fragile. The risk that we saw in our own planning, that the combination of research, architecture, and implementation would be a lot of work, did partly come true. The detection part was finished and tested well, but there was not enough time left to finish the response part. In the end we chose to prioritise the detection pipeline, which was also the core of the MVP.

7 Conclusions

7.1 Conclusions per Research Question

In this section we answer the research questions one by one, based on our results and the validation. We also say clearly where the answer is only partial.

Sub question 1: combining eBPF, runtime monitoring and deep packet inspection. This question is answered. The system runs four sensors together. Suricata does the deep packet inspection, Zeek does the network security monitoring, Falco does the runtime detection, and Tetragon works on the kernel level and can also block. They all run on the nodes and send their data into one pipeline. The validation in Section 5.4 showed that this combination really detects threats, both on the network and inside the containers, so the different types of visibility work together like we wanted.

Sub question 2: automatic response to detected threats. This question is partly answered. On the runtime layer there is a real automatic response, because Tetragon kills a set of attacker tools in the kernel the moment they start, and we tested this live. On the orchestration layer the SOAR platform is deployed and connected to Wazuh, and it actively receives the alerts, tens of thousands of them. The response actions themselves are built and tested: we made AWS Lambda functions that isolate a compromised instance

or strip attacker permissions, and we verified that they work by invoking them against test targets. What is not finished is the last link, turning on the SOAR playbooks so that an incoming alert triggers the matching response on its own. During the project this automatic chain was not activated, so the cross-system response still had to be started by hand. So the answer is that automatic response already works on the runtime layer, and the response actions exist and were tested, but the fully automatic orchestrated response across the whole system is the remaining step.

Sub question 3: how the defense layers should be structured. This question is answered. We split the environment into two separate networks, so the monitoring backend is kept away from the workloads it watches. Inside the cluster the sensors run on every node, and every part of the cluster has its own network policy that says what it is allowed to talk to. This gives a layered defense, from the network segmentation all the way down to a single workload, which is the structure we made in Section 5.2.

Sub question 4: examining the attack surface and validating the solution. This question is answered. We looked at the attack surface with a purple team exercise and ran a list of real attacks against the live environment, all mapped to the MITRE ATT&CK framework. This gave us a clear picture of what is detected, blocked, or missed, and it also found real problems that we could fix afterwards. So the solution was validated in a structured way, and we honestly know where its blind spots are.

Sub question 5: correlating HIDS and NIDS data. This question is answered on the central level. All the data from the host sensors and the network sensors comes together in Wazuh, where it is decoded, matched against our own rules, and shown on one dashboard. So the place where everything is correlated exists and works. What is still basic is the correlation logic itself, because most of the rules only look at one source. Combining events from different sensors into one smarter alert is something that could still be developed further.

Main research question The main question asked how an advanced hybrid monitoring system, with HIDS and NIDS and dynamic network control, can be designed and built to protect RobotLab from advanced attacks. Based on our results, we can say that this is possible. We designed a reference architecture and built a working version on AWS that combines host and network detection, central correlation, threat intelligence, and runtime enforcement, for a predictable monthly cost. The dynamic network control that the question calls SDN is provided by the eBPF based plugin Cilium, instead of a classic SDN controller, for the reasons we gave in Section 4.1.4. Two parts of the question are not fully covered. The automatic response is only partly there, with runtime enforcement and tested response actions in place but the orchestrated playbook chain not yet activated,

and the on premise side could not be tested because it was out of scope. So the system answers the main question for the AWS environment and proves the concept, while the full automated response and the on premise environment remain as work for the future.

7.2 Next Steps and Future Work

The validation and the security testing did not only show what works, they also gave us a clear list of what should be done next. We grouped these next steps into a few themes.

Finishing the automatic response. The most direct next step comes from sub question 2. The pieces of the response layer are already there: the SOAR platform is running and receiving the alerts, and the Lambda response functions were built and tested. The remaining step is to connect them into one automatic chain. The team should turn on a small set of SOAR playbooks, configure the response assets so the playbooks have something to act through, and link them to the most important Wazuh alerts, so that an incoming alert triggers the matching Lambda response on its own. On the Wazuh side the active response feature can also be turned on, so that a clear threat directly triggers an action like blocking an IP or killing a process. This would change the system from strong detection with tested response actions into a fully automatic detection and response loop.

Repairing the cloud monitoring. During the test we found that the AWS account logs (CloudTrail and GuardDuty) were not actually being read into Wazuh, so the cloud part of the system was not monitored. The detection rules for this layer are already written, so the next step is mostly to fix the ingestion so those rules finally get data. After that, attacks against the cloud account, like changing a security group or turning off GuardDuty, would raise an alert.

Closing the blind spots. The network sensors cannot see encrypted traffic, so command and control hidden inside normal web traffic is missed. The best way to deal with this is more prevention than detection: the team should lock down the outgoing traffic of the containers with an allow list, so a connection to an unknown destination is just blocked. Adding DNS logging at the cluster DNS server and a layer of deeper inspection would help too. The runtime enforcement should also be made stronger, so it cannot be bypassed by simply renaming or copying a tool.

Making the system more robust. At the moment the whole SIEM runs on one machine, which is a single point of failure, and during the test the agents even lost their connection for a while. A future version should make the central monitoring highly available, so the detection does not stop if one machine goes down. It would also be good

to archive the alerts to long term storage, so older data is not lost and can be used for compliance.

Hardening and remaining gaps. The security test found several points that should be fixed before going to production. These include encrypting the channel between the agents and the SIEM, fixing the certificate check on the threat intelligence sync, lowering the permissions of the build pipeline and the SOAR so they only have what they need, making the Kubernetes API private instead of open to the internet, and tuning the noisy rules that create too many false alerts. We wrote all of these down in our gap list so they can be picked up one by one.

Ideas for future projects. Next to fixing the open points, there are a few directions we think are interesting for a next project. The first is smarter correlation: right now most rules look at one source, but combining events from different sensors into one alert would make the detection more accurate, which links back to sub question 5. The second is extending the system to the future on premise environment of RobotLab, which was out of scope for us but fits the portable open source stack we chose. The third is doing a longer measurement of the false positives during normal use, so the rules can be tuned with real data. These steps would build further on the foundation this project delivered.

List of Sources

References

- [1] ICT Research Methods, “The DOT Framework,” HBO-i. [Online]. Available: <https://ictresearchmethods.nl/>
- [2] National Institute of Standards and Technology, *NIST SP 800-137: Information Security Continuous Monitoring (ISCM)*, NIST, 2011.
- [3] National Institute of Standards and Technology, *NIST SP 800-94: Guide to Intrusion Detection and Prevention Systems*, NIST, 2007.
- [4] National Institute of Standards and Technology, *NIST SP 800-190: Application Container Security Guide*, NIST, 2017.
- [5] National Institute of Standards and Technology, *NIST SP 800-82 Rev. 3: Guide to Operational Technology (OT) Security*, NIST, 2023.
- [6] P. Cichonski, T. Millar, T. Grance, and K. Scarfone, “Computer Security Incident Handling Guide,” NIST SP 800-61 Rev. 2, 2012. doi: 10.6028/NIST.SP.800-61r2.
- [7] Center for Internet Security, *CIS Critical Security Controls Version 8*, CIS, 2021.
- [8] International Organization for Standardization, *ISO/IEC 27035-1:2016 – Information Security Incident Management – Part 1: Principles of Incident Management*, ISO, 2016.
- [9] International Organization for Standardization, *ISO/IEC 27001:2022 – Information Security Management Systems – Requirements*, ISO, 2022.
- [10] International Electrotechnical Commission, *IEC 62443 Series – Industrial Automation and Control Systems Security*, IEC, 2018–2023.
- [11] MITRE Corporation, *MITRE ATT&CK: Adversarial Tactics, Techniques, and Common Knowledge*. [Online]. Available: <https://attack.mitre.org>
- [12] Gartner, “Market Guide for Extended Detection and Response,” 2022. [Online]. Available: <https://www.gartner.com/en/documents/4015009>
- [13] Gartner, “Market Guide for Security Orchestration, Automation and Response Solutions,” Gartner Research, 2023. [Online]. Available: <https://www.gartner.com/reviews/market/security-orchestration-automation-and-response-solutions>
- [14] Verizon, “2023 Data Breach Investigations Report,” Verizon Business, 2023. [Online]. Available: <https://www.verizon.com/business/resources/reports/dbir/>

- [15] Elastic, “Elastic Security: SIEM, Endpoint Security, and Cloud Security,” 2023. [Online]. Available: <https://www.elastic.co/security>
- [16] Splunk, “Splunk SOAR Documentation,” 2023. [Online]. Available: <https://docs.splunk.com/Documentation/SOAR>
- [17] A. L. Buczak and E. Guven, “A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection,” *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016. doi: 10.1109/COMST.2015.2494502.
- [18] Open Information Security Foundation (OISF), “Suricata User Guide.” [Online]. Available: <https://docs.suricata.io/>
- [19] The Zeek Project, “Zeek Documentation.” [Online]. Available: <https://docs.zeek.org/>
- [20] A. Nadler, A. Aminov, and A. Shabtai, “Detection of Malicious and Low Throughput Data Exfiltration over the DNS Protocol,” *Computers & Security*, vol. 80, pp. 36–53, 2019. doi: 10.1016/j.cose.2018.09.006.
- [21] B. Anderson, S. Paul, and D. McGrew, “Deciphering Malware’s Use of TLS (Without Decryption),” *Journal of Computer Virology and Hacking Techniques*, vol. 14, no. 3, pp. 195–211, 2018. doi: 10.1007/s11416-017-0306-6.
- [22] eBPF Community, “What is eBPF?,” 2024. [Online]. Available: <https://ebpf.io/what-is-ebpf/>
- [23] Isovalent / Cilium Authors, “Cilium Documentation,” 2024. [Online]. Available: <https://docs.cilium.io/en/stable/>
- [24] Cilium Authors, “Hubble – Network Observability for Kubernetes,” 2024. [Online]. Available: <https://docs.cilium.io/en/stable/observability/hubble/>
- [25] Kubernetes Authors, “Network Policies – Kubernetes Documentation,” 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/services-networking/network-policies/>
- [26] The Falco Project, “Falco Documentation,” CNCF. [Online]. Available: <https://falco.org/docs/>
- [27] Cilium Authors, “Tetragon Documentation.” [Online]. Available: <https://tetragon.io/docs/>
- [28] J. A. Donenfeld, “WireGuard: Next Generation Kernel Network Tunnel,” in *Proc. NDSS*, 2017. [Online]. Available: <https://www.wireguard.com/papers/wireguard.pdf>

- [29] Wazuh, “Architecture – Getting started with Wazuh.” [Online]. Available: <https://documentation.wazuh.com/current/getting-started/architecture.html>
- [30] Wazuh, “wazuh/wazuh: Wazuh – The Open Source Security Platform – GitHub.” [Online]. Available: <https://github.com/wazuh/wazuh>
- [31] S. A. Chamkar, M. Zaydi, Y. Maleh, and N. Gherabi, “Improving Threat Detection in Wazuh Using Machine Learning Techniques,” *Journal of Cybersecurity and Privacy*, vol. 5, no. 2, art. 34, 2025. doi: 10.3390/jcp5020034.
- [32] D. Kreutz et al., “Software-Defined Networking: A Comprehensive Survey,” *Proc. IEEE*, vol. 103, no. 1, pp. 14–76, 2015. doi: 10.1109/JPROC.2014.2371999.
- [33] S. Scott-Hayward, S. Natarajan, and S. Sezer, “A Survey of Security in Software Defined Networks,” *IEEE Communications Surveys & Tutorials*, vol. 18, no. 1, pp. 623–654, 2016. doi: 10.1109/COMST.2015.2453114.
- [34] C. Wagner, A. Dulaunoy, G. Wagener, and A. Iklody, “MISP: The Design and Implementation of a Collaborative Threat Intelligence Sharing Platform,” in *Proc. 2016 ACM Workshop on Information Sharing and Collaborative Security*, pp. 49–56. doi: 10.1145/2994539.2994542.
- [35] MISP Project, “MISP Core Software – GitHub Repository.” [Online]. Available: <https://github.com/MISP/MISP>
- [36] Security Onion Solutions, “Security Onion Documentation.” [Online]. Available: <https://docs.securityonion.net/>
- [37] Filigran, “OpenCTI: Open Cyber Threat Intelligence Platform,” 2023. [Online]. Available: <https://www.opencti.io>
- [38] Amazon Web Services, “Amazon GuardDuty Documentation.” [Online]. Available: <https://docs.aws.amazon.com/guardduty/>
- [39] Amazon Web Services, “Amazon Elastic Kubernetes Service Documentation.” [Online]. Available: <https://docs.aws.amazon.com/eks/>
- [40] Amazon Web Services, “AWS CloudTrail Documentation.” [Online]. Available: <https://docs.aws.amazon.com/cloudtrail/>

A Project Plan / Research Plan

The full Research Plan written at the start of the project is delivered as a separate document alongside this report. It contains the original problem statement, the research questions, the scope, the DOT framework approach, the planning, and the risk analysis. It is referred to in Chapter 3 and Chapter 6.

B Individual Tool Research Reports

During the literature research we studied the candidate tools in depth and wrote a separate research document per tool. These documents are the source material for Chapter 4 and are delivered as separate files alongside this report.

Document

XDR – Approaches, Tools and Design Choices

XDR vs SOAR

SDN – Approaches, Tools and Design Choices

Why SDN Is Not the Silver Bullet for Cloud

Cilium

Falco

Tetragon

Zeek

Suricata

Wazuh Manager

Wazuh Indexer

Wazuh Agent

Wazuh Dashboard

Elastic Security

MISP

Security Onion

C Comparative Analyses per Tool Category

The comparative analysis of the candidate tools per category (summarised in Chapter 4.1.6) is delivered as a separate document.

D Comparative Analysis vs. Security Onion + MISP

The full comparative analysis between our stack and the Security Onion + MISP distribution, including the MITRE ATT&CK coverage scoring referenced in Chapter 4.1.6, is delivered as a separate document.

E Attack Simulation Run Book and Detection-Coverage Table

This appendix documents the validation in full. The attack simulation was run as a purple team exercise against the live environment: the attacker executed each technique from a foothold pod and from the external surface, while the defender watched the dashboards and recorded the result. The attack scripts are in `new-infra/red-team/` and the victim-machine installers in `new-infra/scripts/victim-*.sh`. Each technique below lists the MITRE ATT&CK identifier, the command or action used, the sensor expected to react, and the recorded verdict.

Technique	MITRE	Command / action	Result
Run nmap / nc in a container	T1059	<code>nmap -Pn -p80 <ip> ; nc -zw2 <ip> 5432</code>	Blocked (SIGKILL)
Spawn a shell in a container	T1059.004	<code>kubectl exec ... - /bin/sh</code>	Detected
Read SA-token and exfiltrate	T1552.001	<code>read /var/run/secrets/... + curl</code>	Detected
Privilege escalation (setuid)	T1548	<code>setuid/capset to root</code>	Detected
Container escape attempt	T1611	<code>runc exploit (version check)</code>	Blocked (patched)
Pod steals cloud credentials	T1552.005	<code>curl 169.254.169.254/...</code>	Blocked (Cilium)
Port scan from a pod	T1046	<code>nmap -sS -O <ip></code>	Detected
HTTP login brute force	T1110	<code>repeated POST /login</code>	Detected
Denial of service floods	T1498	<code>nping SYN/UDP/ICMP floods</code>	Detected

Technique	MITRE	Command / action	Result
C2 on an odd port	T1071	<code>curl http://<ip>:4444</code>	Blocked + alarmed
DNS tunneling from a pod	T1048.003	<code>nslookup <b64>.exfil...</code>	Detected
Persistence via CronJob	T1053.007	<code>kubectl create cronjob ...</code>	Detected
Lateral movement (app tiers)	T1210	<code>connect db/auth/search</code>	to Visible, not blocked
Encrypted over 443	C2 T1071.001	TLS beacon on 443	Missed
Cloud control-plane events	T1562	SG change / GuardDuty disable	Missed

The screenshots that evidence these results (the SIEM overview, the Tetragon SIGKILL, the Suricata scan/brute/DoS alerts, the Hubble flow map, and the DNS and CronJob detections) are delivered as separate image files alongside this report.

F Pre-Production Gap Register

ID	Severity	Gap
GAP-001	High	Wazuh agent to manager channel (TCP 1514) is unencrypted.
GAP-002	High	MISP sync sidecars use <code>curl -k</code> ; the certificate is not validated.
GAP-003	High	No container image signing or admission control.
GAP-004	Medium	Wazuh runs as a single all-in-one node (single point of failure).
GAP-005	Medium	SOAR instance hardening and access (default credentials, broad IAM) needs review.

GAP-006	Medium	Automated response not fully active. Runtime enforcement (Tetragon) and the Lambda response functions are built and tested, but the SOAR playbooks that trigger them automatically are not yet switched on, so end-to-end response is not yet automatic.
GAP-007	Medium	No certificate rotation plan once GAP-001 and GAP-002 are fixed.
GAP-008	Medium	No long-term archiving of alerts to S3.

G Network and Architecture Diagrams

The full-size network and architecture diagrams are delivered as separate image files. Smaller versions appear in Chapter 5.